



INTEGRATION OF PHYSICS-INFORMED NEURAL NETWORKS IN ALTERNATING SCHWARZ METHODS

done by

Jean-Claude S. MITCHOZOUNOU

COLLEGE OF COMPUTING, UM6P, MOROCCO

MASTER OF QUANTITATIVE AND FINANCIAL MODELLING (QFM)

PROJECT OF THIRD SEMESTER

Supervisor: **PROF. LAHCEN LAAYOUNI**

Academic Year 2024-2025

CONTENTS

1	Introduction	1
1.0.1	Background	1
1.0.2	Motivation and Objective	1
2	Alternating Schwarz Methods and Physics-Informed Neural Networks	2
2.0.1	Overviews on Neural Network	2
2.0.2	Physics-Informed Neural Networks (PINNs) for Solving PDEs	5
2.0.3	The Alternating Schwarz Method	9
2.1	Alternating Schwarz Method - PINNs in $1D$	11
2.1.1	Test Scenarios 1: Dirichlet-type transmission conditions.	12
2.1.2	Test Scenarios 2: Robin-type transmission conditions.	14
2.2	Alternating Schwarz Method - PINNs in $2D$	16
2.2.1	Test Scenarios 3: Dirichlet-type transmission conditions.	17
2.2.2	Test Scenarios 4: Robin-type transmission conditions.	19
3	Conclusion	23

Introduction

1.0.1 Background

The Alternating Schwarz Method (ASM) [2] is a domain decomposition technique used to solve partial differential equations (PDEs) by iteratively solving subproblems on overlapping subdomains. This method is particularly useful for large-scale problems where direct solution methods are computationally expensive. The ASM works by dividing the original domain into smaller subdomains, solving the PDEs in these subdomains independently, and then iterating to achieve convergence across the entire domain.

Recently, the integration of Physics-Informed Neural Networks (PINNs) [1] with ASM has gained attention. PINNs leverage neural networks to approximate solutions to PDEs while incorporating physical laws as constraints in the learning process. By embedding the ASM within the PINNs framework, one can efficiently solve complex PDEs with high accuracy. This hybrid approach enhances the capability of PINNs by incorporating domain decomposition, which allows for parallel computation and improved convergence rates.

In this context, the Alternating Schwarz Method with PINNs provides a powerful tool for addressing a wide range of scientific and engineering problems. By combining the strengths of both methods, researchers can tackle large-scale PDEs more effectively.

1.0.2 Motivation and Objective

The motivation behind combining the Alternating Schwarz Method with Physics-Informed Neural Networks stems from the need to solve high-dimensional and complex PDEs more efficiently. Traditional numerical methods often face challenges such as high computational costs and difficulties in handling complex geometries. The ASM offers a structured way to decompose the problem, making it more manageable, while PINNs provide a flexible and powerful approach to approximate solutions by embedding physical knowledge into the neural network training process. This combination aims to leverage the strengths of both methods to achieve faster convergence, better scalability, and higher accuracy in solving PDEs.

Our objective is to understand the behavior of alternating Schwarz methods when integrated with PINN models in other way, we're trying to propose a framework for embedding PINNs within Schwarz methods and then evaluate the framework's performance against traditional methods.

Alternating Schwarz Methods and Physics-Informed Neural Networks

2.0.1 Overviews on Neural Network

Artificial Neural Network(ANN) is a computational network (a system of nodes and the interconnection between nodes) inspired by biological neural networks, which are the complex networks of neurons in human brains (see Figure 2.1). The nodes created in the ANN are supposedly programmed to behave like actual neurons, and hence they are artificial neurons.

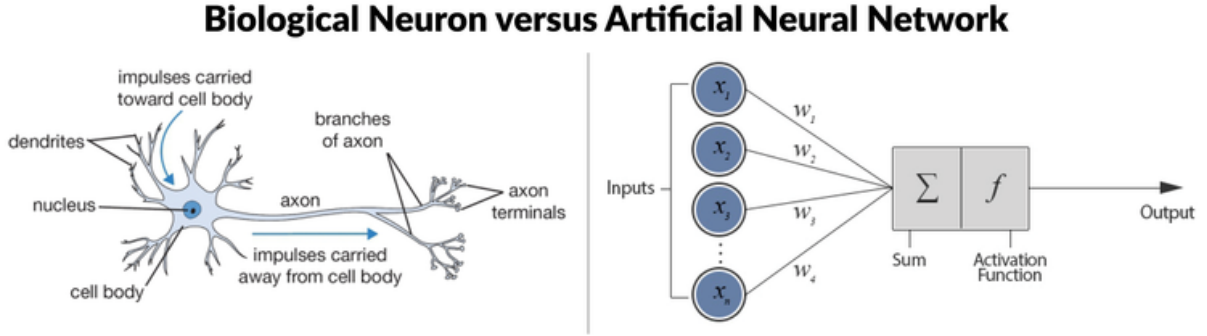


Figure 2.1: Biological Neuron vs. Artificial Neural Network Source: [ResearchGate](#)

Deep Neural Networks

Mathematically, a deep neural network is a particular choice of a compositional function. The simplest neural network is the feedforward neural network (FNN), also called a multilayer perceptron (MLP), which applies linear and nonlinear transformations to the inputs recursively. Although many different types of neural networks have been developed in the past decades, such as the convolutional neural network and the recurrent neural network.

Let $\mathcal{N}_L(x) : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ be an L -layer neural network, or an $(L - 1)$ -hidden layer neural network, with N_ℓ neurons in the ℓ th layer ($N_0 = d_{\text{in}}, N_L = d_{\text{out}}$). Let us denote the weight matrix and bias vector in the ℓ th layer by $\mathbf{W}_\ell \in \mathbb{R}^{N_\ell \times N_{\ell-1}}$ and $\mathbf{b}_\ell \in \mathbb{R}^{N_\ell}$, respectively. Given a nonlinear activation function σ , which is applied element-wise, the FNN is recursively defined as follows:

- **Input layer:** $\mathcal{N}_0(x) = x \in \mathbb{R}^{d_{\text{in}}}$,
- **Hidden layers:** $\mathcal{N}_\ell(x) = \sigma(\mathbf{W}_\ell \mathcal{N}_{\ell-1}(x) + \mathbf{b}_\ell) \in \mathbb{R}^{N_\ell}$, $1 \leq \ell \leq L - 1$,
- **Output layer:** $\mathcal{N}_L(x) = \mathbf{W}_L \mathcal{N}_{L-1}(x) + \mathbf{b}_L \in \mathbb{R}^{d_{\text{out}}}$.

See also a visualization of a neural network in Figure 2.2.

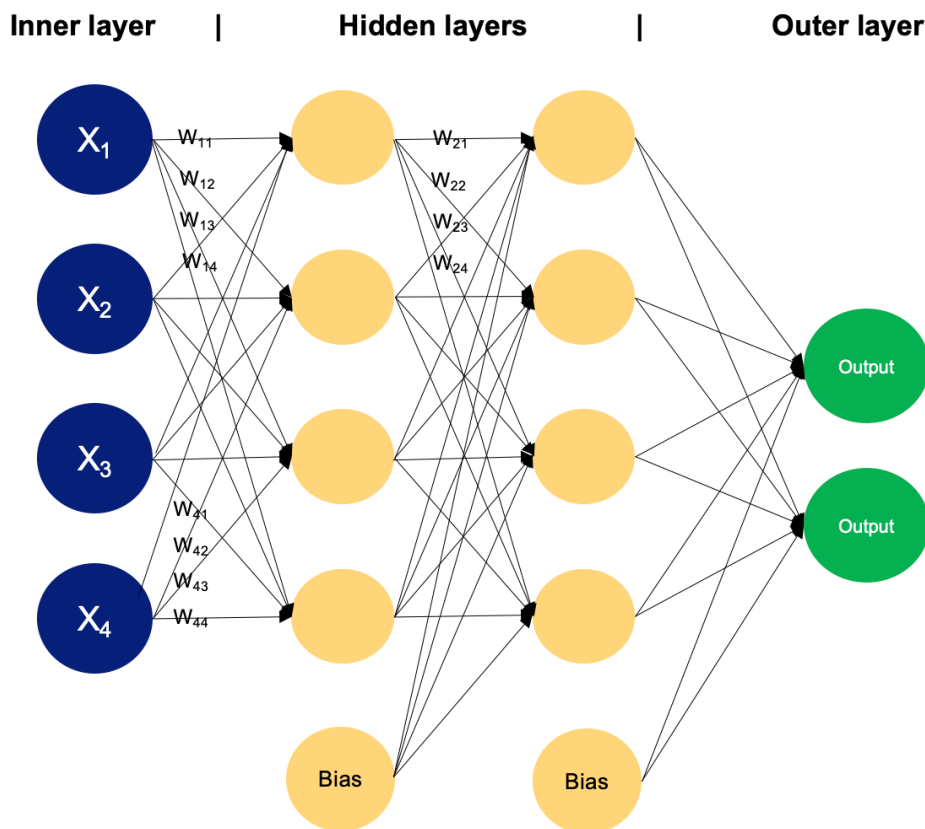


Figure 2.2: Neural Network with two hidden layers Source: [DataCamp](#)

Input Layer

The input layer contains neurons corresponding to the dimensions of the input data. Each neuron represents a feature, and the number of neurons matches the data's dimensionality.

Hidden Layer

Hidden layers lie between the input and output layers. Each neuron in a hidden layer connects to all neurons in the previous layer and passes its output to the next. The number of hidden layers and neurons are hyperparameters decided during model design.

Output Layer

The output layer produces the final result. Its number of neurons depends on the task: one or two for binary classification (depending on activation) and multiple for multi-class tasks.

Weights

Connections between neurons in adjacent layers have weights, which determine their strength and are learned during training.

Bias Neurons

Bias neurons, present in all layers except the input, provide constant input and have learnable weights, enhancing the model's flexibility.

Activation Functions: The idea of an activation function comes from the analysis of how a neuron works in the human brain (see Figure 2.3). The neuron becomes active beyond a

certain threshold, better known as the activation potential. These functions introduce nonlinearity into the network, allowing it to learn complex patterns in the data. It also attempts to put the output into a small range in most cases.

Commonly used activation functions include the logistic sigmoid $1/(1 + e^{-x})$, the hyperbolic tangent (tanh), and the rectified linear unit (ReLU, $\max\{x, 0\}$).

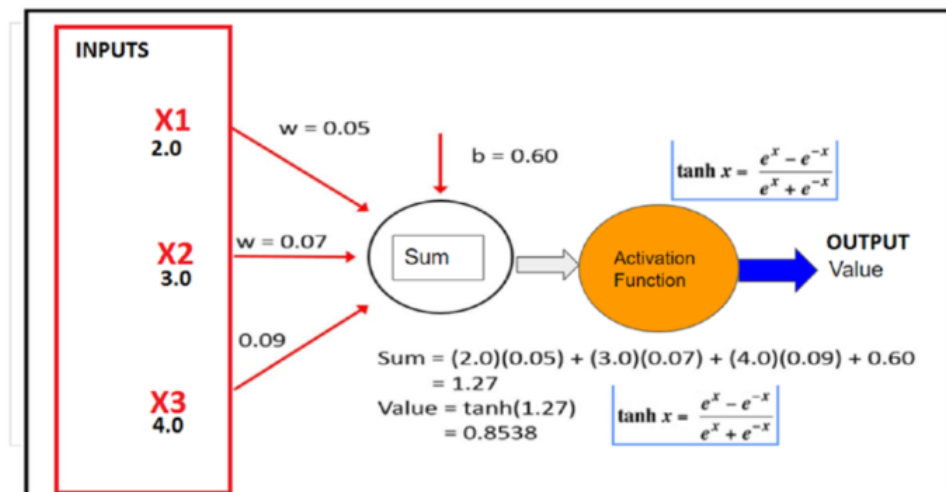


Figure 2.3: An activation function

Loss Functions: The loss function (cost function) is to be minimized so as to get the best values for each parameter of the model. For any model, there are numerous parameters, and the model structure in prediction or classification is expressed in terms of the values of the parameters. We need to evaluate your model, and for that we need to define the cost function (loss function). The *minimization of the loss function* can be the driving force for finding the optimum value of each parameter.

Optimizers: Now we should be convinced that we need to use a loss function to get the best value of each parameter of the model. How can we get the best value?

Initially we assume the initial values of weight and bias for the model (linear regression, etc.). Now we need to find the way to reach to the best value of the parameters. The optimizer is the way to reach the best value of the parameters. In each iteration, the value changes in a direction suggested by the optimizer

Metrics: Having learned some ways to build a model, it is time to evaluate the model. So, we need to evaluate the regressor or classifier. There are many evaluation metrics, among which classification accuracy, logarithmic loss, and area under ROC curve are the most popular ones.

Remark 2.1. All these approaches employ machine learning algorithms like support vector machines, random forests, Gaussian processes, and feed-forward/convolutional/recurrent neural networks merely as black-box tools.

Automatic Differentiation

Automatic differentiation in general, and the back-propagation algorithm in particular, is currently the dominant approach for training deep models by taking their derivatives with respect to the parameters (e.g., weights and biases) of the models. Indeed, MLPs are trained using the backpropagation algorithm, a specialized technique of AD which computes gradients of a

loss function with respect to the model's parameters and updates the parameters iteratively to minimize the loss.

Considering the fact that the neural network represents a compositional function, AD applies the chain rule repeatedly to compute the derivatives. There are two steps in AD: one forward pass to compute the values of all variables, and one backward pass to compute the derivatives. To demonstrate AD, we consider an FNN of only one hidden layer with two inputs, x_1 and x_2 , and one output, y :

$$\begin{aligned} v &= -2x_1 + 3x_2 + 0.5, \\ h &= \tanh(v), \\ y &= 2h - 1. \end{aligned}$$

The forward pass and backward pass of automatic differentiation (AD) for computing the partial derivatives $\frac{\partial y}{\partial x_1}$ and $\frac{\partial y}{\partial x_2}$ at $(x_1, x_2) = (2, 1)$ are shown in Table 2.1.

Table 2.1: Example of AD to compute the partial derivatives $\frac{\partial y}{\partial x_1}$ and $\frac{\partial y}{\partial x_2}$ at $(x_1, x_2) = (2, 1)$.

Forward pass	Backward pass
$x_1 = 2$	$\frac{\partial y}{\partial y} = 1$
$x_2 = 1$	
$v = -2x_1 + 3x_2 + 0.5 = -0.5$	$\frac{\partial y}{\partial h} = \frac{\partial(2h-1)}{\partial h} = 2$
$h = \tanh v \approx -0.462$	$\frac{\partial y}{\partial v} = \frac{\partial y}{\partial h} \cdot \frac{\partial h}{\partial v} = \frac{\partial y}{\partial h} \cdot \text{sech}^2(v) \approx 1.573$
$y = 2h - 1 = -1.924$	$\frac{\partial y}{\partial x_1} = \frac{\partial y}{\partial v} \cdot \frac{\partial v}{\partial x_1} = \frac{\partial y}{\partial v} \cdot (-2) \approx -3.146$
	$\frac{\partial y}{\partial x_2} = \frac{\partial y}{\partial v} \cdot \frac{\partial v}{\partial x_2} = \frac{\partial y}{\partial v} \cdot 3 \approx 4.719$

We can see that AD requires only one forward pass and one backward pass to compute all the partial derivatives, no matter what the input dimension is. In contrast, using finite differences to compute each partial derivative $\frac{\partial y}{\partial x_i}$ requires two function evaluations $y(x_1, \dots, x_i, \dots, x_{d_{\text{in}}})$ and $y(x_1, \dots, x_i + \Delta x_i, \dots, x_{d_{\text{in}}})$ for some small number Δx_i . Thus, in total, $d_{\text{in}} + 1$ forward passes are required to evaluate all the partial derivatives. Hence, AD is much more efficient than finite differences when the input dimension is high.

To compute n th-order derivatives, AD can be applied recursively n times. However, this nested approach may lead to inefficiency and numerical instability, and hence other methods, e.g., Taylor-mode AD, have been developed for this purpose. Finally, we note that with AD we differentiate the neural network directly and, therefore, can deal with noisy data.

In PINNs, we will see that it is required to compute the derivatives of the network outputs with respect to the network inputs (see Table 2.1). This allows us to open the black-box by understanding and appreciating the key role played by automatic differentiation within the deep learning field.

2.0.2 Physics-Informed Neural Networks (PINNs) for Solving PDEs

The Challenge of Machine Learning in Scientific Domains

With the rapid expansion of data availability and computing power, machine learning (ML) and data analytics have revolutionized numerous scientific fields, including image recognition, cognitive science, and genomics. However, when it comes to complex physical, biological, or engineering systems, this progress is often constrained by the high cost and difficulty of data acquisition. Consequently, researchers frequently face the challenge of drawing conclusions and

making decisions based on incomplete or limited information.

At the same time, a common theme across scientific domains is the disparity between the rapid generation of observational data and the slower ability to assimilate and interpret it meaningfully. Although ML approaches demonstrate remarkable empirical promise and have achieved preliminary successes, they often fall short in extracting interpretable information and actionable knowledge from this "data deluge." Moreover, purely data-driven models, while capable of fitting observations effectively, frequently produce predictions that are physically inconsistent or implausible leading to poor generalization.

The Need for Physics-Informed Machine Learning

To address these challenges, there is a pressing need to integrate fundamental physical laws and domain knowledge into machine learning models. Unlike conventional ML approaches that rely solely on observational data, physics-informed learning incorporates governing physical principles to guide the learning process. This approach introduces "informative priors"—theoretical constraints and inductive biases derived from known physical rules—on top of observational data. By embedding these priors, physics-informed ML models can improve robustness, ensure physical consistency, and enhance generalization, ultimately unlocking their potential to analyze and predict complex systems more reliably.

It should be noted that generally, when we use classical machine learning to deal with physical phenomena, we often encounter three scenarios about categories of physical problems and associated available data. These three scenarios can be illustrated by following figure.

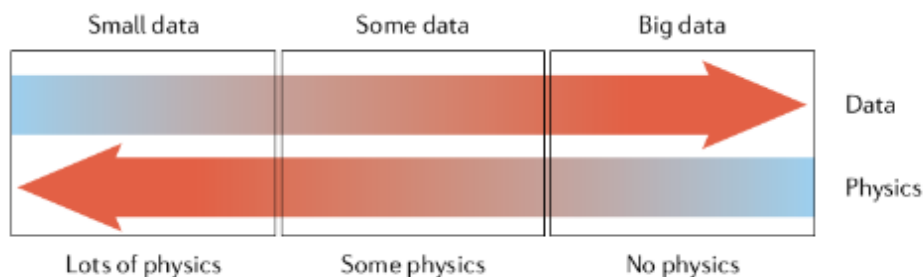


Figure 2.4: Data and Physics scenarios

- In the small data regime, it is assumed that one knows all the physics, and data are provided for the initial and boundary conditions as well as the coefficients of a partial differential equation;
- The ubiquitous regime in applications is the middle one, where one knows some data and some physics, possibly missing some parameter values or even an entire term in the partial differential equation, for example, reactions in an advection–diffusion–reaction ;
- The regime with big data is the one, where one may not know any of the physics, and where a data-driven approach may be most effective, for example, using operator regression methods to discover new physics.

Physics-informed machine learning can seamlessly integrate data and the governing physical laws, including models with partially missing physics, in a unified way. This can be expressed compactly using automatic differentiation and neural networks that are designed to produce predictions that respect the underlying physical principles

Concepts and mechanism of PINNs

We consider the following PDE parameterized by $\boldsymbol{\lambda}$ for the solution $u(x)$ with $x = (x_1, \dots, x_d)$ defined on a domain $\Omega \subset \mathbb{R}^d$:

$$f\left(x; \frac{\partial u}{\partial x_1}, \dots, \frac{\partial u}{\partial x_d}; \frac{\partial^2 u}{\partial x_1^2}, \dots, \frac{\partial^2 u}{\partial x_1 \partial x_d}; \dots; \boldsymbol{\lambda}\right) = 0, \quad x \in \Omega, \quad (2.1)$$

with suitable boundary conditions

$$\mathcal{B}(u, x) = 0 \quad \text{on } \partial\Omega, \quad (2.2)$$

where $\mathcal{B}(u, x)$ could represent Dirichlet, Neumann, Robin, or periodic boundary conditions. For time-dependent problems, we consider time t as a special component of x , and Ω includes the temporal domain. The initial condition can be treated as a special type of Dirichlet boundary condition on the spatio-temporal domain.

In a Physics-Informed Neural Network (PINN), we first construct a neural network $u(x; \hat{\boldsymbol{\theta}})$ as a surrogate for the solution $u(x)$, which takes the input x and outputs a vector with the same dimension as u . Here, $\hat{\boldsymbol{\theta}} = \{\mathbf{W}_\ell, \mathbf{b}_\ell\}_{1 \leq \ell \leq L}$ represents the set of all weight matrices and bias vectors in the neural network $\hat{u}$.

One advantage of PINNs, by choosing neural networks as the surrogate for u , is that we can compute the derivatives of $\hat{u}$ with respect to its input x by applying the chain rule for differentiating compositions of functions using automatic differentiation (AD). This process is conveniently integrated into popular machine learning packages such as TensorFlow and PyTorch.

In the next step, we need to restrict the neural network $\hat{u}$ to satisfy the physics imposed by the PDE and boundary conditions. In practice, we restrict $\hat{u}$ on some scattered points (e.g., randomly distributed points or clustered points in the domain), i.e., the training data $\mathcal{T} = \{x_1, x_2, \dots, x_{|\mathcal{T}|}\}$ of size $|\mathcal{T}|$. Additionally, $\mathcal{T}$ comprises two subsets, $\mathcal{T}_f \subset \Omega$ and $\mathcal{T}_b \subset \partial\Omega$, representing points in the domain and on the boundary, respectively. We refer to $\mathcal{T}_f$ and $\mathcal{T}_b$ as the sets of "residual points."

To measure the discrepancy between the neural network $\hat{u}$ and the constraints, we define a loss function as the weighted summation of the L_2 norm of residuals for the equation and boundary conditions:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = w_f \mathcal{L}_f(\boldsymbol{\theta}; \mathcal{T}_f) + w_b \mathcal{L}_b(\boldsymbol{\theta}; \mathcal{T}_b), \quad (2.3)$$

where

$$\mathcal{L}_f(\boldsymbol{\theta}; \mathcal{T}_f) = \frac{1}{|\mathcal{T}_f|} \sum_{x \in \mathcal{T}_f} \left\| f\left(x; \frac{\partial \hat{u}}{\partial x_1}, \dots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1^2}, \dots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \dots; \boldsymbol{\lambda}\right) \right\|_2^2, \quad (2.4)$$

and

$$\mathcal{L}_b(\boldsymbol{\theta}; \mathcal{T}_b) = \frac{1}{|\mathcal{T}_b|} \sum_{x \in \mathcal{T}_b} \|\mathcal{B}(\hat{u}, x)\|_2^2. \quad (2.5)$$

Here, w_f and w_b are weights. The loss function involves derivatives, such as the partial derivative $\partial \hat{u} / \partial x_1$ or the normal derivative at the boundary $\partial \hat{u} / \partial n = \nabla \hat{u} \cdot \mathbf{n}$, which are computed using automatic differentiation (AD).

In the final step, the procedure of searching for an optimal $\boldsymbol{\theta}$ by minimizing the loss $\mathcal{L}(\boldsymbol{\theta}; \mathcal{T})$ is referred to as "training."

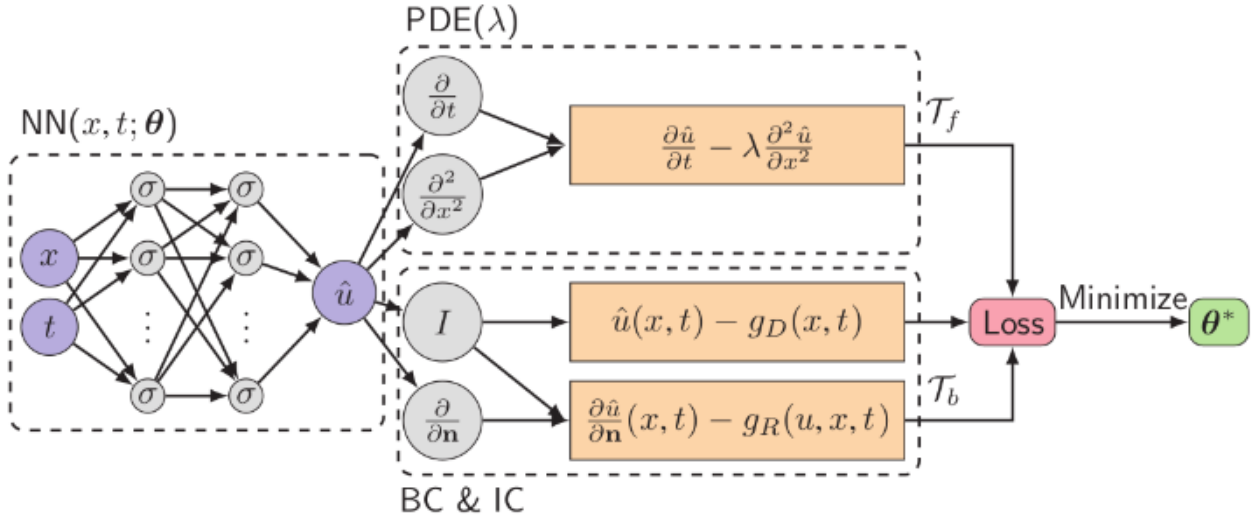


Figure 2.5: PINNs Achitecture

Algorithm 1 The PINN algorithm for solving differential equations.

- 1: **Step 1:** Construct a neural network $u(x; \hat{\theta})$ with parameters θ .
 - 2: **Step 2:** Specify the two training sets $\mathcal{T}_f$ and $\mathcal{T}_b$ for the equation and boundary/initial conditions.
 - 3: **Step 3:** Specify a loss function by summing the weighted L_2 norm of both the PDE equation and boundary condition residuals.
 - 4: **Step 4:** Train the neural network to find the best parameters θ^* by minimizing the loss function $\mathcal{L}(\theta; \mathcal{T})$.
-

Lack of uniqueness

Unlike traditional numerical methods, PINNs do not guarantee unique solutions because they solve nonconvex optimization problems, which generally lack uniqueness. Achieving good accuracy requires careful tuning of hyperparameters such as network size, learning rate, and the number of residual points. The network size depends on the smoothness of the PDE solution: smaller networks suffice for simpler problems like the 1D Poisson equation, while more complex PDEs like the 1D Burgers equation require deeper and wider networks. Indeed, uniform sampling may not efficiently capture regions with high dynamics.

Since PINNs may converge to different solutions depending on the initialization, a common approach is to train the network multiple times (e.g., 10 runs) and select the model with the lowest training loss.

Adaptive residual points

In PINNs, the residual points $\mathcal{T}$ are typically randomly distributed in the domain. While this approach works well for many cases, it may not be efficient for certain PDEs with solutions that exhibit steep gradients. For example, in the case of the Burgers equation, it is intuitive to place more residual points near the sharp front to better capture the discontinuity. However, designing an optimal distribution of residual points is challenging when the solution is unknown.

To address this issue, *Lu Lu and al [1]* propose a residual-based adaptive refinement (RAR) method to improve the distribution of residual points during the training process (Procedure

2.2), conceptually similar to finite element method (FEM) refinement techniques [2]. The core idea of RAR is to add more residual points in regions where the PDE residual

$$\left\| f \left(x; \frac{\partial \hat{u}}{\partial x_1}, \dots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1^2}, \dots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \dots; \boldsymbol{\lambda} \right) \right\|_2^2 \quad (2.6)$$

is large. This process is repeated until the mean residual

$$\mathcal{E}_r = \frac{1}{V} \int_{\Omega} \left\| f \left(x; \frac{\partial \hat{u}}{\partial x_1}, \dots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1^2}, \dots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \dots; \boldsymbol{\lambda} \right) \right\|_2^2 dx \quad (2.7)$$

is smaller than a specified threshold $\mathcal{E}_0$, where V is the volume of Ω .

Algorithm 2 RAR for improving the distribution of residual points for training.

- 1: **Step 1:** Select the initial residual points $\mathcal{T}$ and train the neural network for a limited number of iterations.
- 2: **Step 2:** Estimate the mean PDE residual $\mathcal{E}_r$ in (2.3) using Monte Carlo integration, calculated as the average of residual values at a set of randomly sampled dense locations $\mathcal{S} = \{x_1, x_2, \dots, x_{|\mathcal{S}|}\}$:

$$\mathcal{E}_r \approx \frac{1}{|\mathcal{S}|} \sum_{x \in \mathcal{S}} \left\| f \left(x; \frac{\partial \hat{u}}{\partial x_1}, \dots, \frac{\partial \hat{u}}{\partial x_d}; \frac{\partial^2 \hat{u}}{\partial x_1^2}, \dots, \frac{\partial^2 \hat{u}}{\partial x_1 \partial x_d}; \dots; \boldsymbol{\lambda} \right) \right\|_2^2. \quad (2.8)$$

- 3: **Step 3:** Stop if $\mathcal{E}_r < \mathcal{E}_0$. Otherwise, add m new points with the largest residuals in $\mathcal{S}$ to $\mathcal{T}$, retrain the network, and repeat Step 2.
-

2.0.3 The Alternating Schwarz Method

The **Alternating Schwarz Method** is an iterative technique used to solve partial differential equations (PDEs) by decomposing the problem domain into overlapping subdomains. This method is particularly useful for large-scale problems where solving the PDE in the entire domain directly is computationally expensive or impractical.

In one-dimensional space, this method involves solving the PDE alternately in each subdomain while using the solution from the adjacent subdomain as a boundary condition. This ensures the continuity of the solution across the subdomains.

We consider solving the following problem using the alternating Schwarz method, which was introduced in 1870 by Hermann Schwarz in his paper [?]:

$$\begin{cases} (\eta - \Delta)u = f & \text{in } \Omega, \\ u = 0 & \text{on } \partial\Omega, \end{cases} \quad (1)$$

where Ω is a Lipschitz bounded domain in $\mathbb{R}^d$, $d = 1, 2, 3$.

Key Steps in the Alternating Schwarz Method

- **Step 1: Domain Decomposition**

The physical domain Ω is divided into two (or more) overlapping subdomains, Ω_1 and Ω_2 , with an overlapping interface Γ as depicted in Figure 2.6.

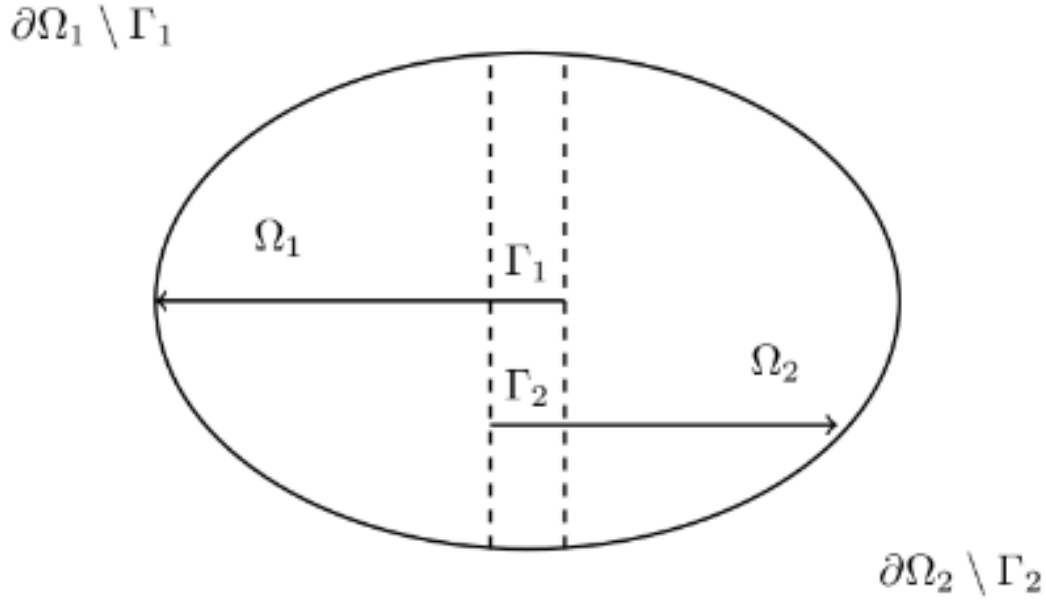


Figure 2.6: Enter Caption

- **Step 2: Iterative Solving**

In this step,

- Start by solving the PDE in one subdomain (e.g., Ω_1) using initial or guessed boundary conditions u_1^0 on the overlap Γ .
- Use the solution from Ω_1 as a boundary condition to solve the PDE in the other subdomain (Ω_2).
- Repeat this process iteratively, exchanging boundary conditions between Ω_1 and Ω_2 as follows:

The solution u_1^n in Ω_1 is computed first using the boundary condition provided by u_2^{n-1} on Γ_1 .

$$\begin{cases} (\eta - \Delta)u_1^n = f_1 & \text{in } \Omega_1, \\ u_1^n = 0 & \text{on } \partial\Omega_1 \setminus \Gamma_1, \\ u_1^n = u_2^{n-1} & \text{on } \Gamma_1 \end{cases} \quad (2) \quad ;$$

Subsequently, the solution u_2^n in Ω_2 is computed using u_1^n on Γ_2

$$\begin{cases} (\eta - \Delta)u_2^n = f_2 & \text{in } \Omega_2, \\ u_2^n = 0 & \text{on } \partial\Omega_2 \setminus \Gamma_2, \\ u_2^n = u_1^n & \text{on } \Gamma_2, \end{cases} \quad (3)$$

until the solution converges across the entire domain.

- **Step 3: Convergence**

The method ensures continuity of the solution across the subdomains, achieving convergence to the global solution of the PDE.

Remark 2.2. In this algorithm, the solution in subdomain Ω_2 at iteration $n+1$ depends on the solution in subdomain Ω_1 also at iteration $n+1$. This contemporaneous dependence prevents one from solving the Ω_1 and Ω_2 problems in parallel.

Remark 2.3. The Schwarz iteration can be modified to achieve what is commonly referred to as additive Schwarz [7] by applying boundary conditions from the n th Schwarz iteration procedure in Ω_1 to the $(n+1)^{st}$ Ω_2 sub-problem. If this change is made, the Schwarz iteration sequence within can be parallelized over the number of subdomains, as done in [8, 9].

2.1 Alternating Schwarz Method - PINNs in 1D

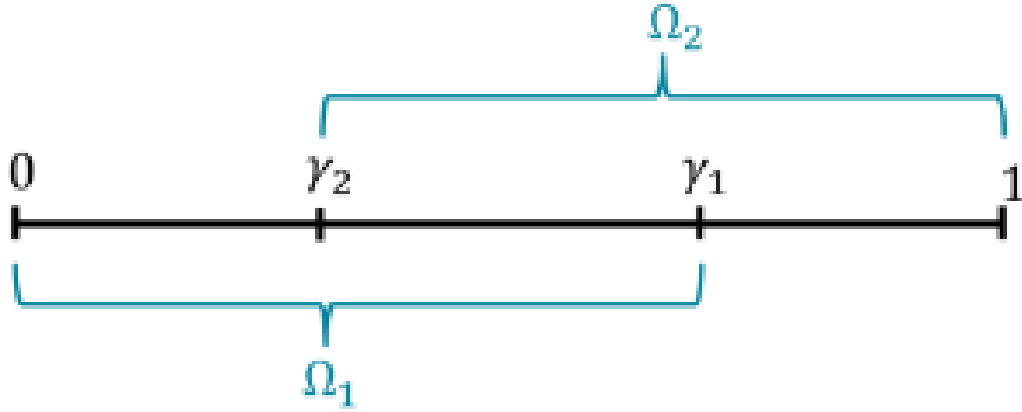


Figure 2.7: Example overlapping domain decompositions for Schwarz alternating method in 1D

First, we start with the one-dimensional case to understand the method's behavior in a simpler context and observe the classical behavior of the alternating Schwarz method when integrating the PINN model. For this purpose, we decompose the interval $I = [-1, 1]$ into two overlapping subdomains: $I_1 = [-1, \delta]$ and $I_2 = [0, 1]$. The overlap is defined by the region $[0, \delta]$, where $\delta > 0$ represents the buffer size between the subdomains.

The corresponding alternating Schwarz method in the one-dimensional domain is given by:

$$\begin{cases} u_1^n - (u_1^n)'' = f_1 & \text{in } I_1, \\ u_1^n(-1) = 0, \\ u_1^n(\delta) = u_2^{n-1}(\delta), \end{cases} \quad \begin{cases} u_2^n - (u_2^n)'' = f_2 & \text{in } I_2, \\ u_2^n(1) = 0, \\ u_2^n(0) = u_1^n(0). \end{cases} \quad (4)$$

Remark 2.4. The points $x_i \in \{-1, 1\}$ for the domain Ω are the coordinates of boundary points at which Dirichlet boundary conditions are prescribed, whereas $x_i \in \{-1, \delta > 0\}$ and $x_i \in \{0, 1\}$ are for the subdomains Ω_1 and Ω_2 , respectively. For 1D subdomains, the maximum number of boundary points for which the weak boundary loss is computed (within each subdomain) is thus $b = 2$.

To integrate the PINN model in solving the alternating Schwarz method, we need to define the loss functions for each subdomain.

In subdomain I_1 , the loss function is given by:

$$L_{I_1} = \frac{1}{N_1} \sum_{i=1}^{N_1} \left[(u_1^n - (u_1^n)'' - f_1)^2 + (u_1^n(-1))^2 + (u_1^n(\delta) - u_2^{n-1}(\delta))^2 \right], \quad (5)$$

where:

- L_{I_1} : Loss for subdomain I_1 ,
- N_1 : Number of training points in subdomain I_1 .

In subdomain I_2 , the loss function is:

$$L_{I_2} = \frac{1}{N_2} \sum_{i=1}^{N_2} \left[(u_2^n - (u_2^n)'' - f_2)^2 + (u_2^n(1))^2 + (u_2^n(0) - u_1^n(0))^2 \right], \quad (6)$$

where:

- L_{I_2} : Loss for subdomain I_2 ,
- N_2 : Number of training points in subdomain I_2 .

Algorithm Description

This algorithm is an iterative process for training and predicting with a model on two subdomains, I_1 and I_2 . The steps are as follows:

1. Initialize a variable $u_2^0(\delta)$ randomly.
2. Enter a loop that continues while the combined loss $L_{I_1} + L_{I_2}$ is greater than a specified tolerance and the number of iterations is less than a maximum limit:
 - (a) Solve for u_1^n in subdomain I_1 using the loss function L_{I_1} .
 - (b) Solve for u_2^n in subdomain I_2 using the loss function L_{I_2} .
 - (c) Update boundary conditions based on u_1^n and u_2^n .
3. Repeat until convergence or reaching the maximum number of iterations.

2.1.1 Test Scenarios 1: Dirichlet-type transmission conditions.

Here our problem (P) can be written as follows:

$$\begin{cases} (\eta - \Delta)u = f & \text{in } [-1, 1], \\ u = 0 & \text{on } \{-1, 1\}, \end{cases} \quad (P_{Diric})$$

The alternating Schwarz method with *PINNs* integrated associated to this problem has been described above in the section 2.2 and lead to Dirichlet-type transmission conditions (see (4)). Here is what we obtain after simulating this algorithm on this problem (P_{Diric})

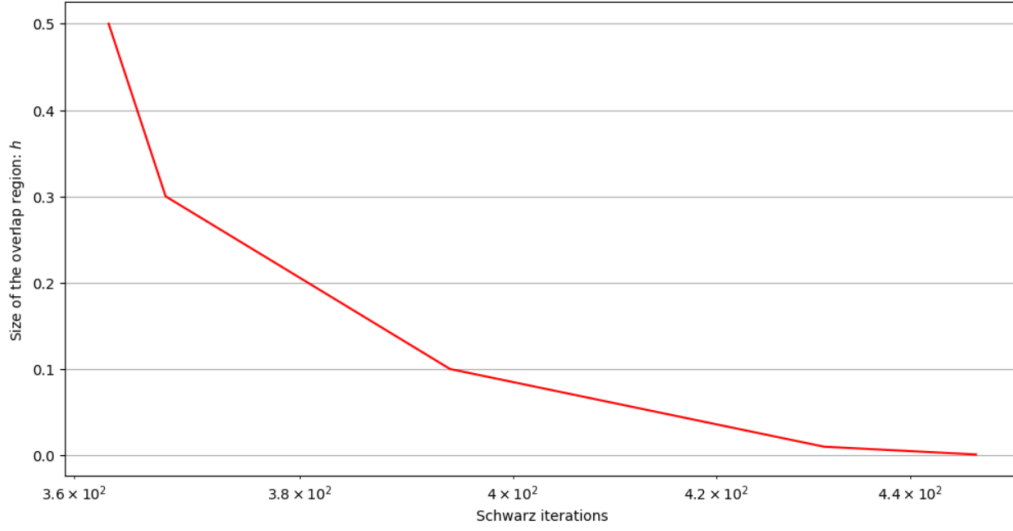


Figure 2.8: Convergence of alternating Schwarz method with ten training points.

By analyzing this plot, we note that as the size of the overlap domain approaches zero, convergence becomes slow as the number of iterations before convergence increases. This involves that the number of Schwarz iterations is almost inversely proportional to the size of the overlap region. The result is less surprising because it has been demonstrated in several references including [6, 8, 9]

In the training we've saved the prediction of the solution on each subdomain at the first and the final iteration. The following plots show us the obtained solution.

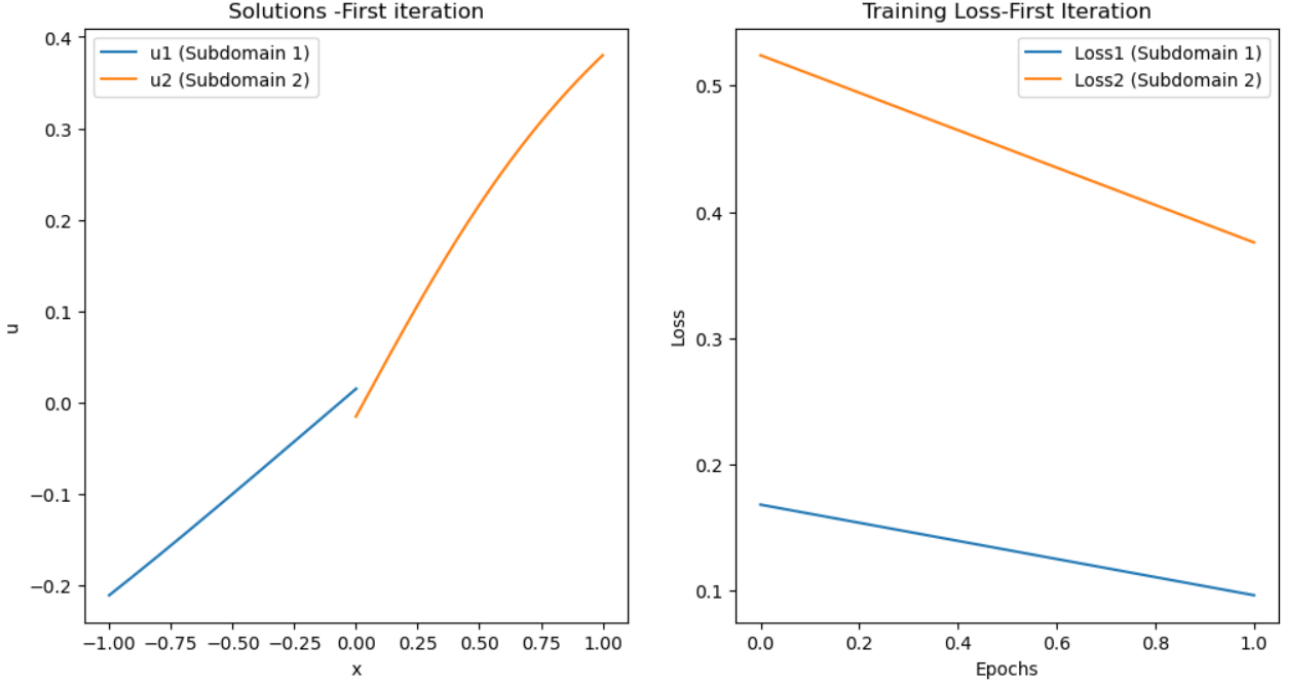


Figure 2.9: Converged solution and loss functions at the first iteration

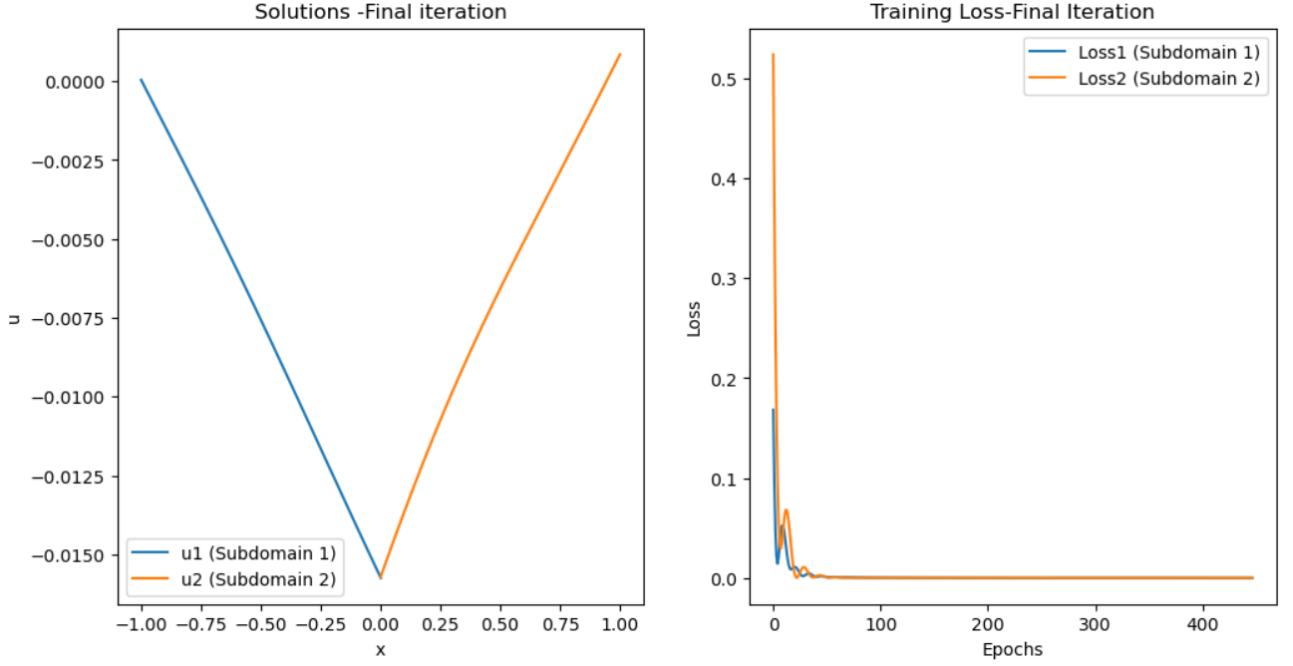


Figure 2.10: Converged solution and loss functions at the final iteration

As the alternating Schwarz's method with Dirichlet-type transmission conditions converges slowly when the overlap becomes small, we need to find a way of speeding up convergence a little. The following section will tell us how to proceed.

2.1.2 Test Scenarios 2: Robin-type transmission conditions.

The classical transmission conditions used by Schwarz are Dirichlet transmission conditions. These transmission conditions lead to convergence iterations which are not uniform with respect to overlap size: large overlap size involves rapid convergence whereas for small overlap size the convergence is slow.

Then, to improve the convergence of the alternating Schwarz method, we use Robin-type transmission conditions. These conditions enhance communication between subdomains by incorporating weighted averages of the solutions and their derivatives at the boundaries. The modified alternating Schwarz method can be written as follows:

$$\begin{cases} \eta u_1^n - (u_1')^n = f_1 & \text{in } I_1, \\ u_1^n(-1) = 0, \\ pu_1^n(\delta) + (u_1')^n(\delta) = pu_2^{n-1}(\delta) + (u_2')^{n-1}(\delta), \end{cases} \quad \begin{cases} u_2^n - (u_2')^n = f_2 & \text{in } I_2, \\ u_2^n(1) = 0, \\ qu_2^n(0) + (u_2')^n(0) = qu_1^n(0) + (u_1')^n(0). \end{cases} \quad (7)$$

So, the loss functions need to be adjusted according to the new transmission conditions as follows:

$$\mathcal{L}_1 = \frac{1}{N_1} \sum_{i=1}^{N_1} \left[(u_1^n - (u_1')^n - f_1)^2 + (u_1(-1))^2 + \left((pu_1^n(\delta) + (u_1')^n(\delta)) - (pu_2^{n-1}(\delta) + (u_2')^{n-1}(\delta)) \right)^2 \right], \quad (8)$$

and

$$\mathcal{L}_2 = \frac{1}{N_2} \sum_{i=1}^{N_2} \left[(u_2^n - (u_2')^n - f_2)^2 + (u_2(1))^2 + \left((qu_2^n(0) + (u_2')^n(0)) - (qu_1^n(0) + (u_1')^n(0)) \right)^2 \right]. \quad (9)$$

We also simulate this algorithm on our problem to really see how the new transmission condition improve the convergence. Note that the new transmission conditions depend of two parameters p and q . We've done a simulation for different values of p and q and different numbers of training points. The goal later will be to determine the optimal values of these parameters. Here is what has been found:

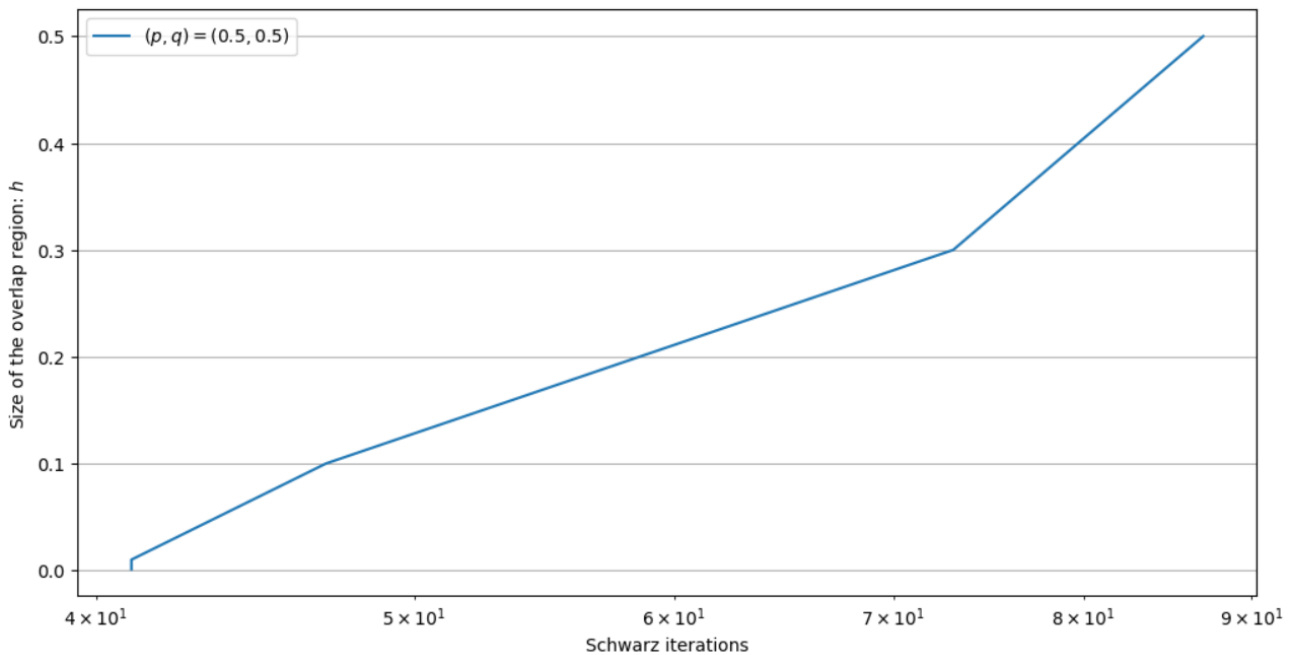


Figure 2.11: Convergence behaviour with ten training points $(p, q) = (0.5, 0.5)$

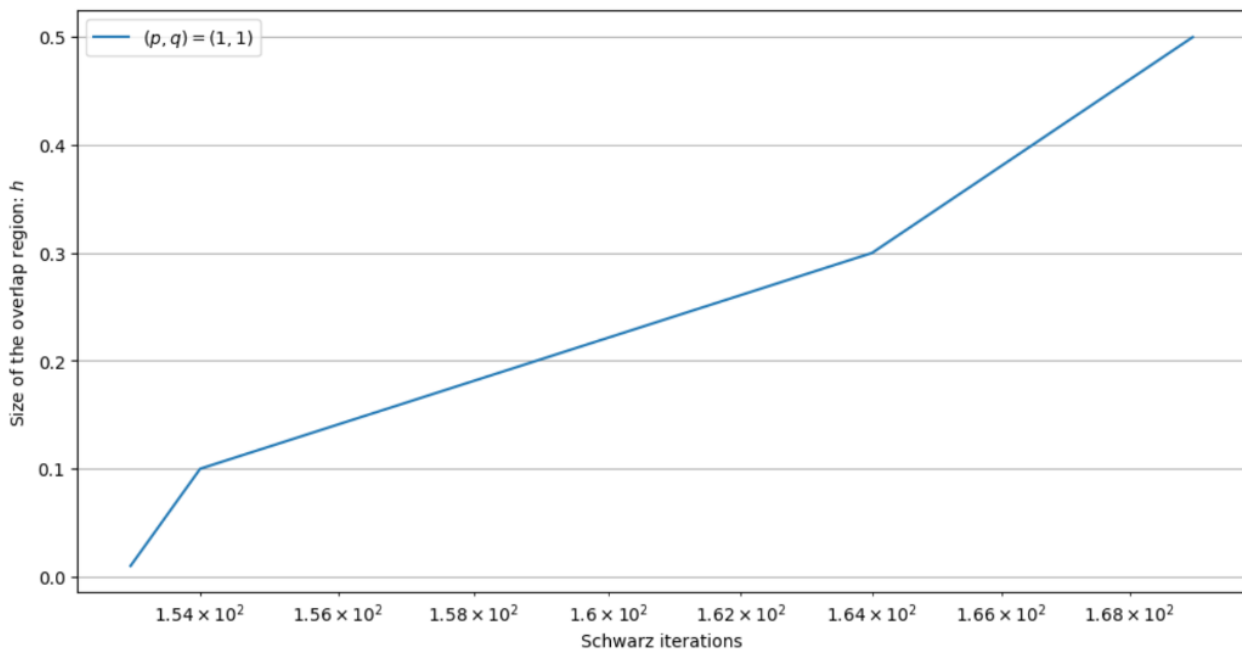


Figure 2.12: Convergence behaviour with ten training points and $(p, q) = (1, 1)$

By analyzing all the previous figures 2.11 and 2.12, we remark that by using the Robin-type transmission conditions,

- the Schwarz iterations have been almost divided by three compared to what we've obtained using Dirichlet-type transmission conditions. This involves that convergence of the alternating Schwarz method has been improved.

- And further, the Schwarz iterations is almost proportional to the size of the overlap region.

In the training process with the news loss functions, we've saved the prediction of the solution on each subdomain at the first and the final iteration. The following plots show us the obtained solution.

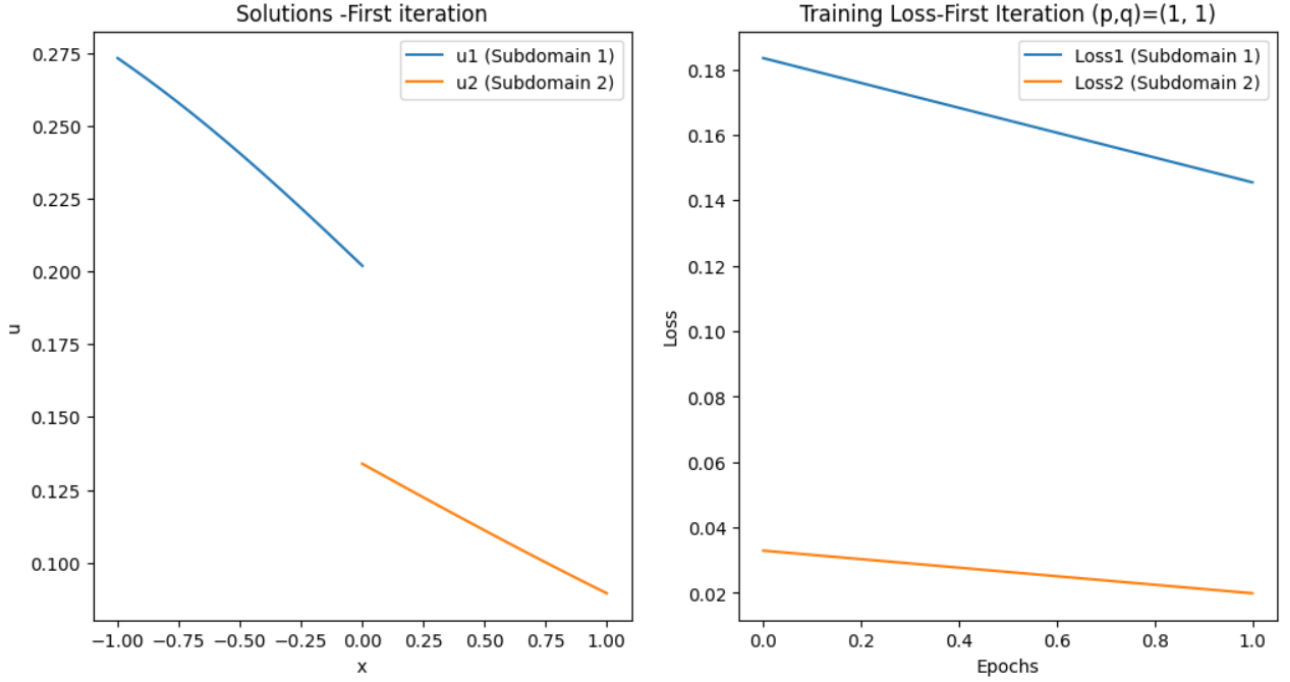


Figure 2.13: Converged solution and Robin-loss functions at the first iteration

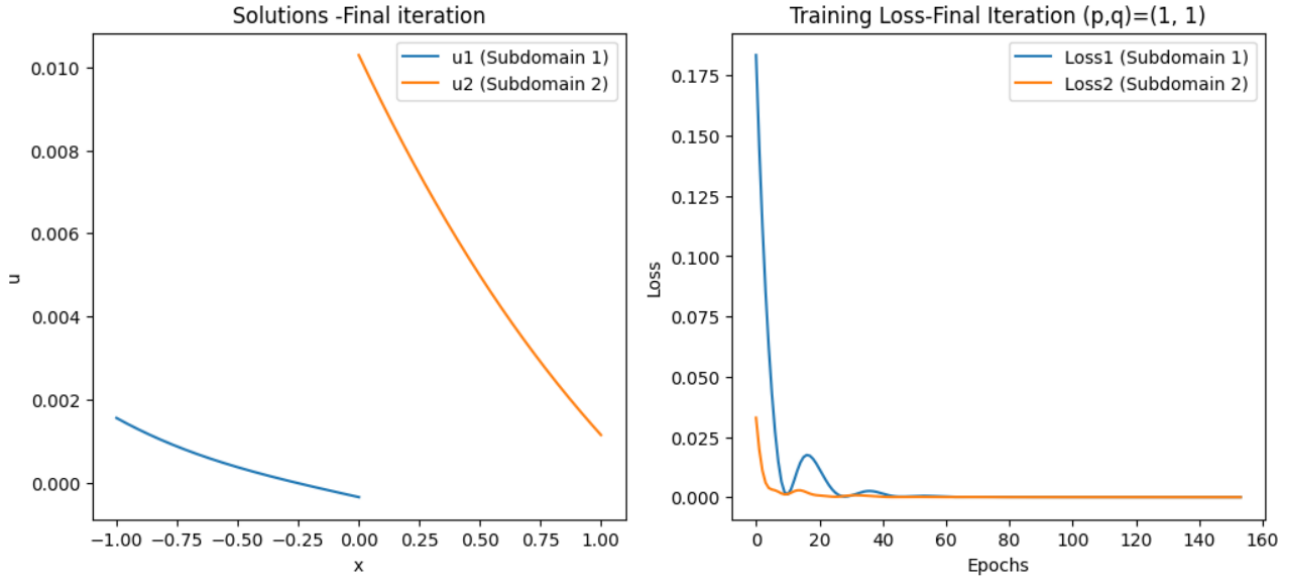


Figure 2.14: Converged solution and Robin-loss functions at the final iteration

2.2 Alternating Schwarz Method - PINNs in 2D

Extending results to the two-dimensional case involves accounting for the interactions across multiple spatial directions. In two dimensions, the normal derivative, $\frac{\partial u}{\partial n}$, must be computed on all boundaries of the rectangular domain, where the outward normal vector varies depending

on the specific edge. This section outlines the extensions of one-dimensional results to the two-dimensional case, ensuring consistent and accurate treatment of boundary conditions across the entire domain.

2.2.1 Test Scenarios 3: Dirichlet-type transmission conditions.

For the sake of simplicity, we consider the plane $([-1, 1])^2$ decomposed into two half-planes $\Omega_1 = [-1, \delta] \times [-1, 1]$ and $\Omega_2 = [0, 1] \times [-1, 1]$ with an overlap of size $\delta > 0$.

We will always consider the problem (1) ($\eta > 0$):

$$(\eta - \Delta)u = f \quad \text{in } \mathbb{R}^2,$$

The Schwarz method for this problem is the following iteration:

- In Ω_1 :

$$\begin{aligned} (\eta - \Delta)u_1^{n+1} &= f(x, y), & (x, y) \in \Omega_1, \\ u_1^{n+1}(\delta, y) &= u_2^n(\delta, y), & y \in [-1, 1]. \end{aligned}$$

- In Ω_2 :

$$\begin{aligned} (\eta - \Delta)u_2^{n+1} &= f(x, y), & (x, y) \in \Omega_2, \\ u_2^{n+1}(0, y) &= u_1^n(0, y), & y \in [-1, 1]. \end{aligned}$$

with the local solutions u_j^{n+1} , $j = 1, 2$.

The integration of the PINN model is the same as previously. The loss function can be constructed samely only by considering the function f_1, f_2, u_1 and u_2 as functions of two variables.

We've implemented and simulated the behavior of ASM-PINNs with Dirichlet transmission conditions as we did in one dimensional case. The following plot shows us how the overlap domain's size influences the convergence of the method.

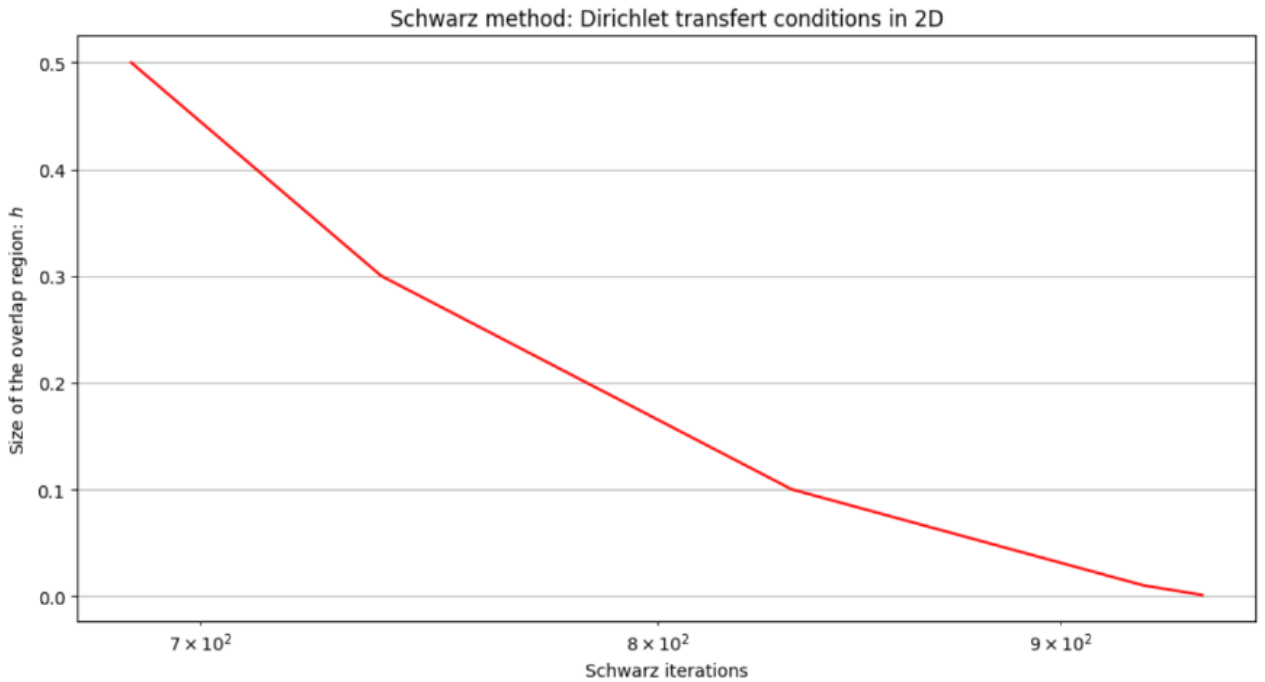


Figure 2.15: Convergence behaviour of Schwarz method with Dirichlet TC

As in 1D, the convergence slows down as the overlap size decreases, with the number of Schwarz iterations being nearly inversely proportional to the overlap size. This happens because the overlap acts as a communication channel between subdomains. A smaller overlap limits the information exchange, providing less accurate boundary conditions and resulting in slower convergence.

Find in the following the display of the predicted solutions using the ASM-PINNs with Dirichlet transmission conditions.

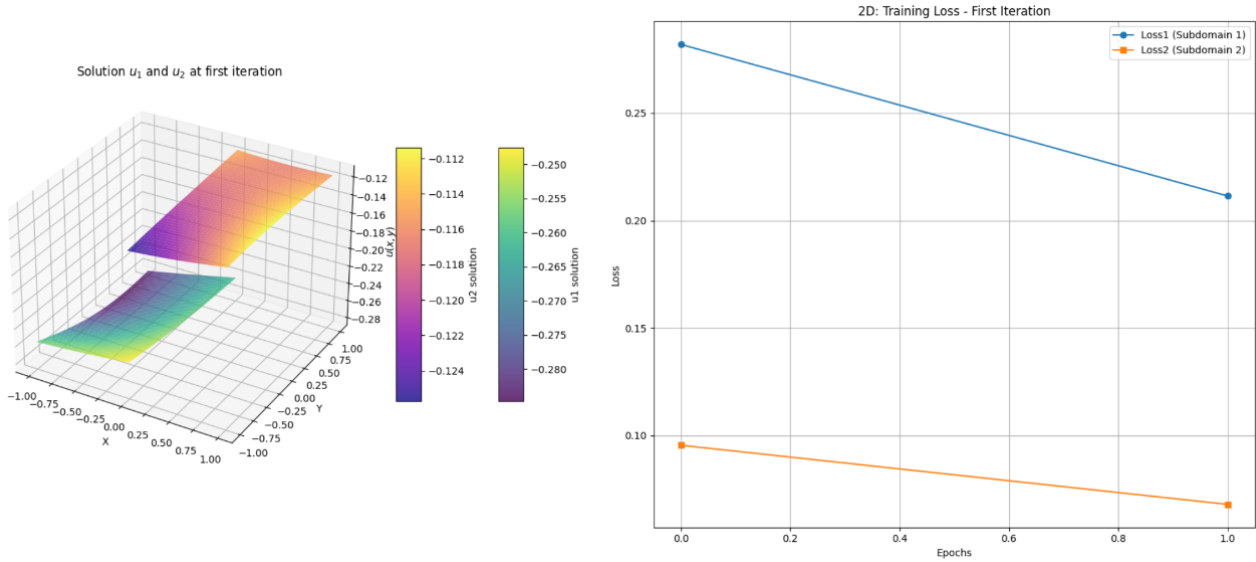


Figure 2.16: Solutions at first iteration with Dirichlet TC

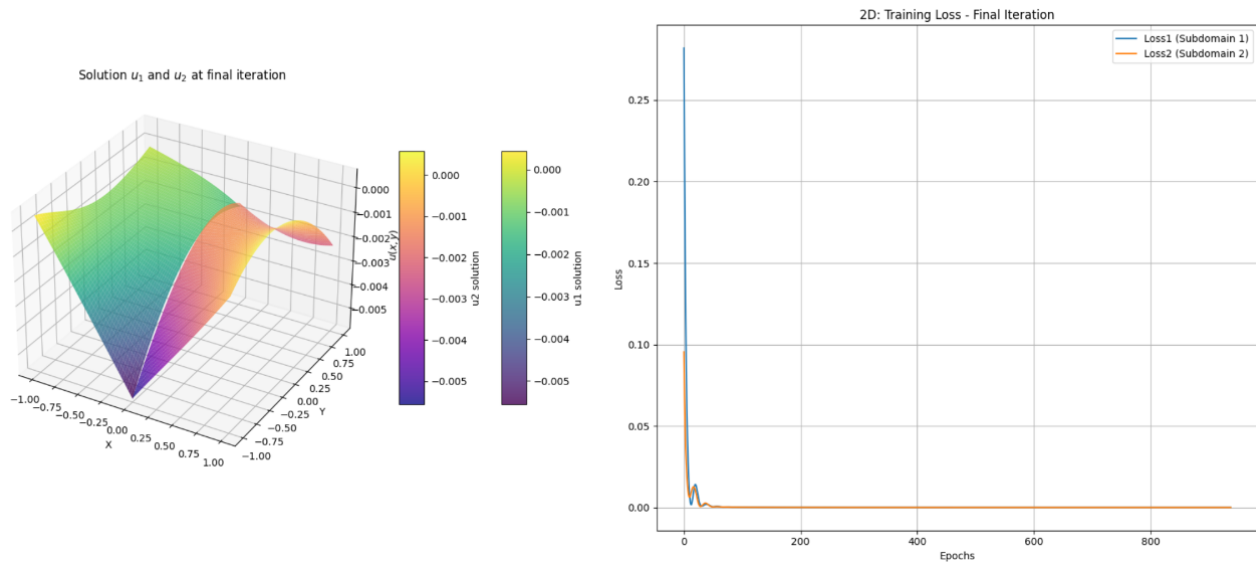


Figure 2.17: Solutions at final iteration with Dirichlet TC

2.2.2 Test Scenarios 4: Robin-type transmission conditions.

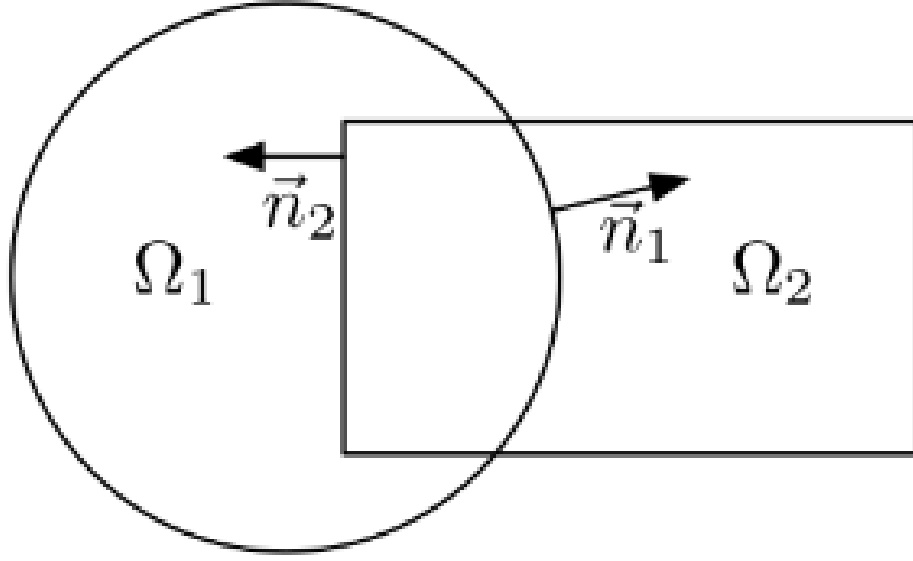


Figure 2.18: Outward normals for overlapping subdomains

For modified Helmholtz equation, we have seen that Schwarz algorithms work overlapping domain and their performance in terms of iterations counts depends on the width of the overlap. Same as in 1D case, it is based on improving Schwarz methods by replacing the Dirichlet interface conditions with Robin interface conditions.

Let p be a positive number, the modified algorithm reads

$$\begin{aligned} (\eta - \Delta)u_1^{n+1} &= f \quad \text{in } \Omega_1, \\ u_1^{n+1} &= 0 \quad \text{on } \partial\Omega_1 \cap \partial\Omega, \\ \frac{\partial u_1^{n+1}}{\partial n_1} + pu_1^{n+1} &= \frac{\partial u_2^n}{\partial n_1} + pu_2^n \quad \text{on } \partial\Omega_1 \cap \Omega_2, \end{aligned} \quad (10)$$

and

$$\begin{aligned} (\eta - \Delta)u_2^{n+1} &= f \quad \text{in } \Omega_2, \\ u_2^{n+1} &= 0 \quad \text{on } \partial\Omega_2 \cap \partial\Omega, \\ \frac{\partial u_2^{n+1}}{\partial n_2} + pu_2^{n+1} &= \frac{\partial u_1^n}{\partial n_2} + pu_1^n \quad \text{on } \partial\Omega_2 \cap \Omega_1, \end{aligned} \quad (11)$$

where n_1 and n_2 are the outward normals on the boundary of the subdomains, see Figure 2.18.

We have also implemented and simulated the behavior of ASM-PINNs with Robin transmission conditions as we did in the one-dimensional case in the other to see how it is better than the Dirichlet one.

The following plot shows us how ASM using Robin transmission conditions outperformed Dirichlet (see figure 2.19). You will also find the predicted solution of our problem using this transmission approach via the figures 2.20 and 2.21 below.

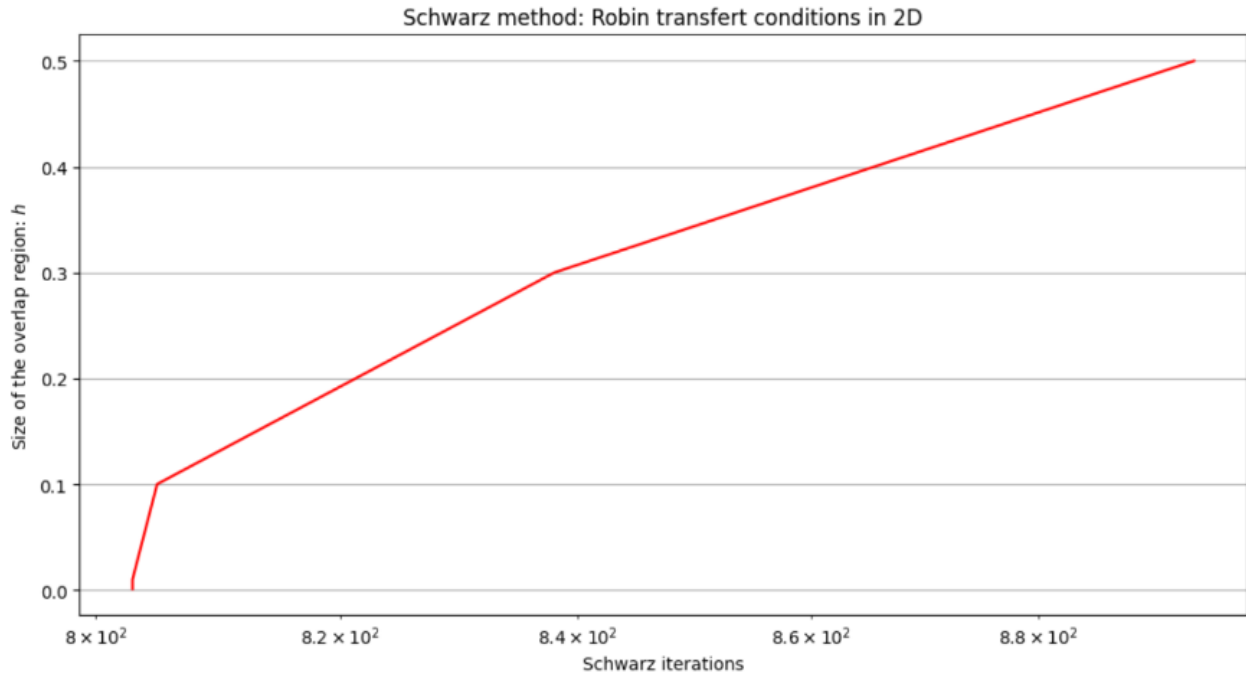


Figure 2.19: Convergence behaviour of Schwarz method with Robin TC

As you can see from this figure, the problem of slow convergence when h becomes small is solved by adopting Robin TC. However, another problem appears : *In this specific case, larger overlap size h appears to hinder convergence (i.e., more iterations are required). This could indicate that the choice of Robin parameters is not optimal for larger overlaps, potentially causing poor information transfer across subdomains*

Indeed, the use of **Robin transmission conditions** may introduce complexities compared to standard Dirichlet or Neumann conditions. Robin conditions depend on a parameter (often a combination of Dirichlet and Neumann terms), and improper tuning of this parameter could lead to counterintuitive behavior, such as slower convergence with increasing overlap.

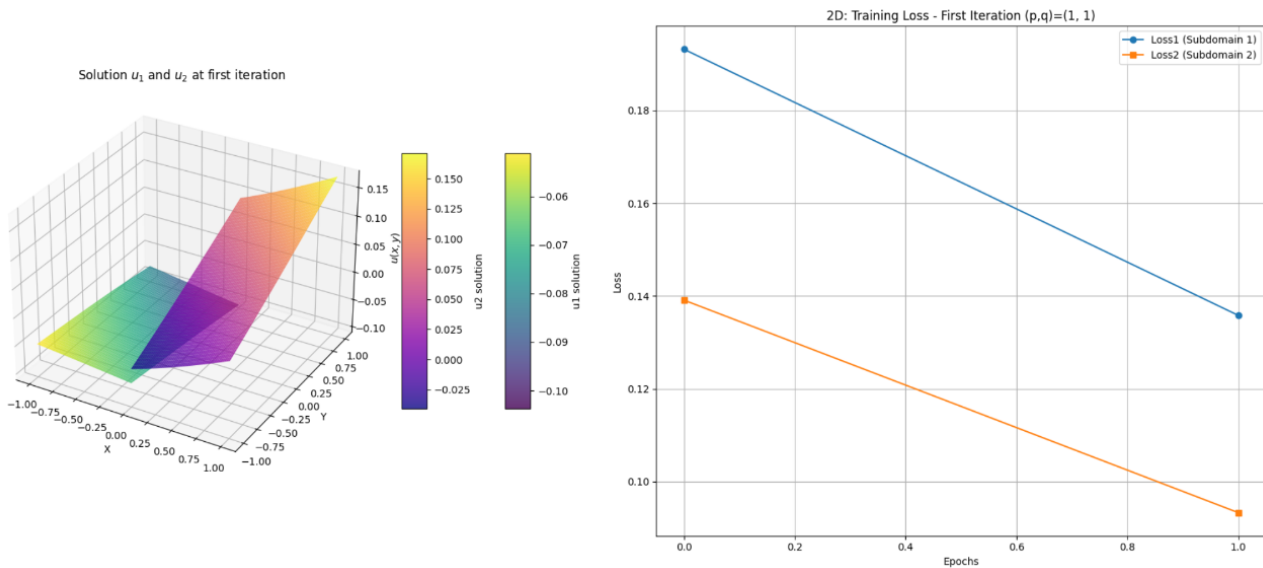


Figure 2.20: Solutions at first iteration with Robin TC

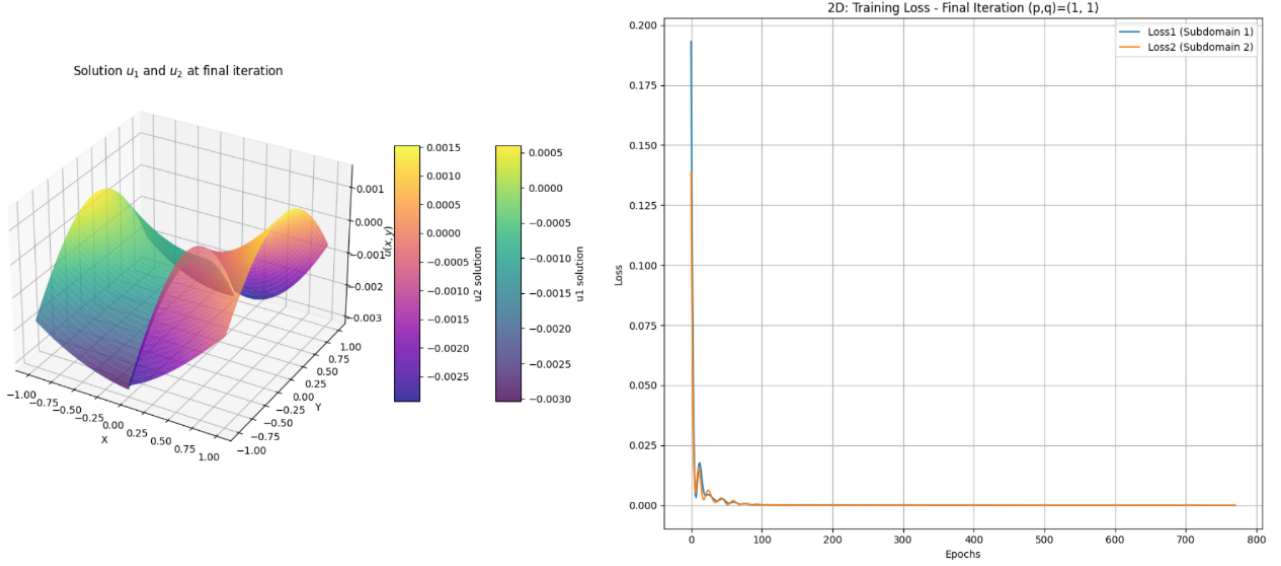


Figure 2.21: Solution in each subdomain at final iteration.

This highlights the need to find the optimum parameters for Robin transmission conditions. Several researchers have analyzed these transmission conditions trying to find the optimal parameter for it. According to Fourier analysis [?], the optimal parameter p^* behaves asymptotically as

$$p^* = \frac{(4(k_{\min}^2 + \eta))^{1/3}}{2} h^{-1/3}, \quad (12)$$

where $k_{\min} = \pi$ for an unbounded domain Ω .

In the following, we have used the optimal value p^* in Robin transmission conditions and then to see how it optimizes

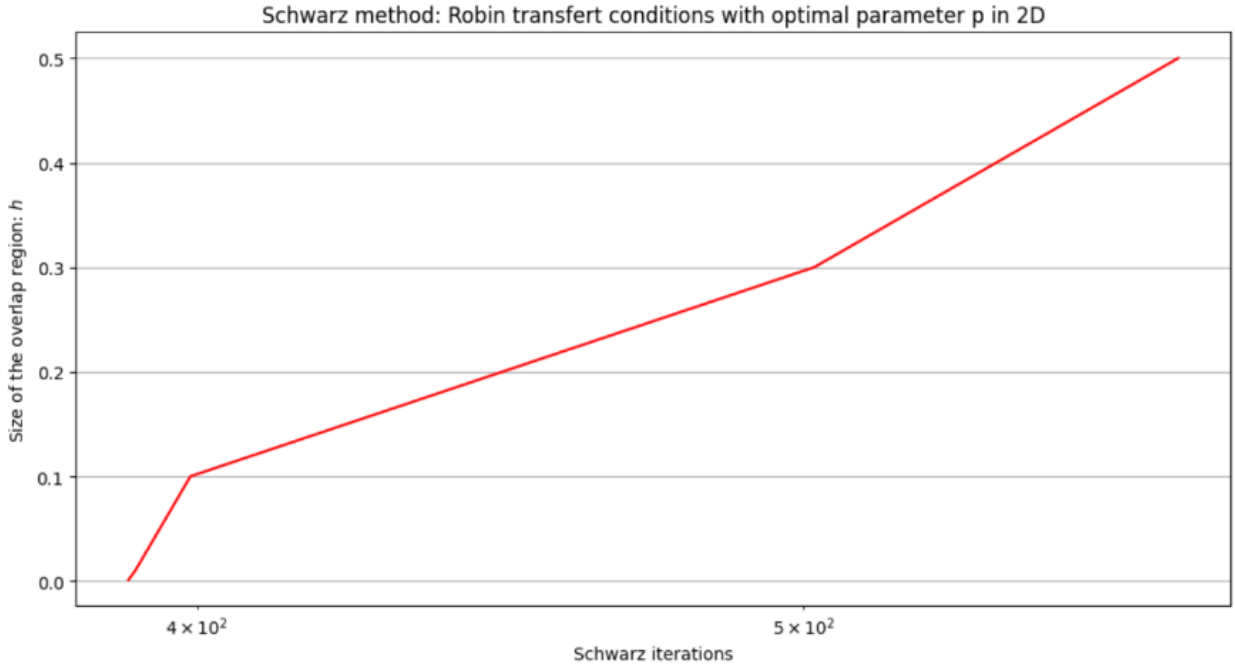


Figure 2.22: Optimized Schwarz iterations

The plot for the optimized relaxation parameter shows a steeper increase in h with respect to the number of Schwarz iterations. The convergence rate is significantly faster, as fewer

iterations are required to achieve the same overlap size or convergence threshold. Indeed, in term of quantitative impact, we can remark from the figures that:

- With $\mathbf{p} = \mathbf{1}$ (see 2.22): It takes approximately **800+ iterations** to achieve a significant overlap size.
- With $\mathbf{p}^*$ (see 2.19): The method converges in **400-500 iterations**, reducing the iteration count by nearly **40-50%**.

This dramatic improvement highlights the importance of tuning the relaxation parameter for optimal performance.

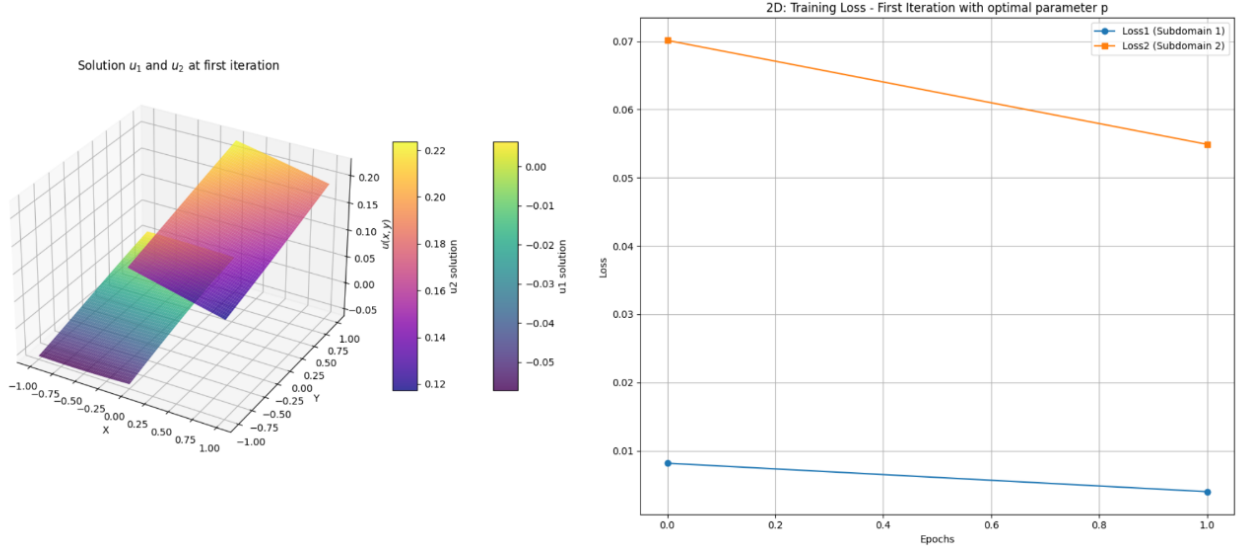


Figure 2.23: Shape of the solution in each subdomain and losses at first iteration.

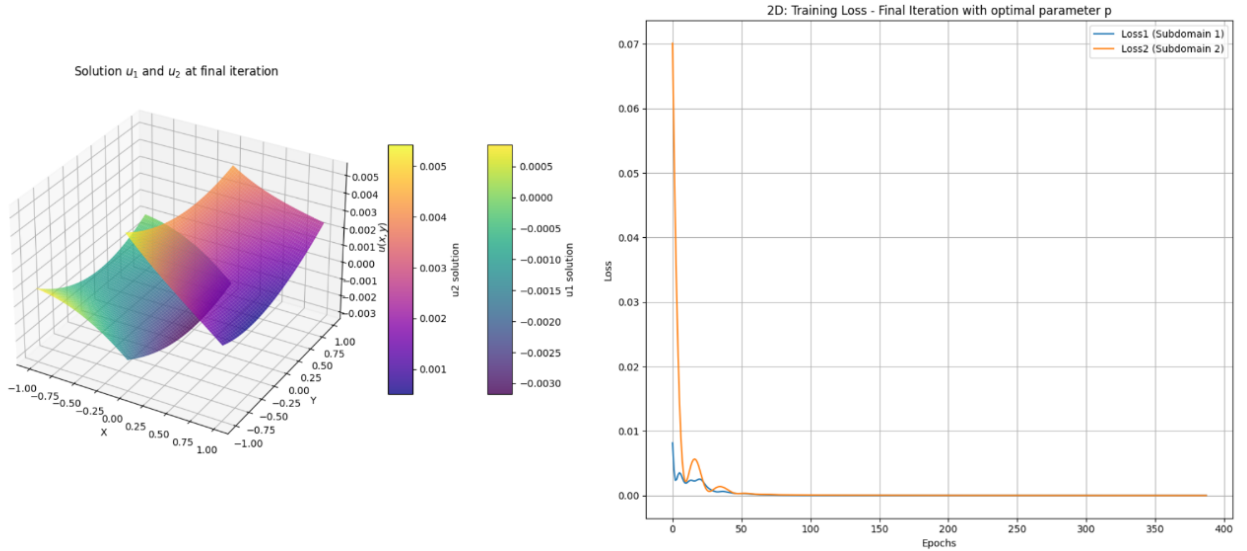


Figure 2.24: Shape of the solution in each subdomain and losses at final iteration.

Using the optimal relaxation parameter p^* significantly accelerates convergence by optimizing the interface conditions between subdomains. This results in fewer Schwarz iterations and better computational efficiency.

Conclusion

This project explored the integration of the **Alternating Schwarz Method (ASM)** with **Physics-Informed Neural Networks (PINNs)**, providing a robust framework for solving partial differential equations (PDEs). The study demonstrated that while classical Dirichlet transmission conditions in ASM are straightforward to implement, they result in slower convergence for smaller overlaps. Replacing these conditions with Robin transmission conditions significantly improved convergence by enhancing communication between subdomains. However, the performance of Robin conditions was shown to depend heavily on the choice of relaxation parameters, with suboptimal values leading to slower convergence for larger overlaps.

The introduction of the **optimal relaxation parameter p^*** , derived from Fourier analysis, dramatically improved the convergence rate, reducing the required iterations by nearly 40-50%. The study demonstrated that optimized transmission conditions not only enhance the efficiency of the ASM but also align well with the flexibility and scalability of PINNs. These findings highlight the importance of parameter tuning and the potential of hybrid approaches like ASM-PINNs for solving complex PDEs efficiently in various scientific and engineering applications.

BIBLIOGRAPHY

- [1] Lu Lu Xuhui Meng Zhiping Mao George Em Karniadakis *DeepXDE: A Deep Learning Library for Solving Differential Equations*
- [2] M. Ainsworth and J. T. Oden, *A Posteriori Error Estimation in Finite Element Analysis*, John Wiley & Sons, 2011. (Cited on p. 217)
- [3] M. Raissi , P. Perdikaris , G.E. Karniadakis , *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*
- [4] George Em Karniadakis, Ioannis G. Kevrekidis^{3,4}, Lu Lu Sifan Wang⁷ and Liu Yang, *Physicsinformed machine learning*
- [5] I. E. Lagaris, A. Likas and D. I. Fotiadis, *Artificial Neural Networks for Solving Ordinary and Partial Di erential Equations*
- [6] WILLIAM SNYDER , IRINA TEZAUR[†] , AND CHRISTOPHER R. WENTLAND, *DOMAIN DECOMPOSITION-BASED COUPLING OF PHYSICS-INFORMED NEURAL NETWORKS VIA THE SCHWARZ ALTERNATING METHOD*
- [7] M. J. Gander, *Schwarz methods over the course of time.*, ETNA. *Electronic Transactions on Numer- ical Analysis [electronic only]*, 31 (2008), pp. 228–255.
- [8] K. Li, K. Tang, T. Wu, and Q. Liao, *D3m: A deep domain decomposition method for partial differential equations*, *IEEE Access*, 8 (2020), pp. 5283–5294.
- [9] W. Li, X. Xiang, and Y. Xu, *Deep Domain Decomposition Method: Elliptic Problems*, *Proceedings of Machine Learning Research*, 107 (2020), pp. 269–286.
- [10] Victorita Dolean, Pierre Jolivet, Frédéric Nataf *An Introduction to Domain Decomposition Methods: algorithms, theory and parallel implementation*
- [11] Martin J. Gander ¹ , Laurence Halpern ² , Frederic Nataf ³ *Optimized Schwarz Methods*

Codes

Here the link to access the project on github <https://github.com/JCPY6235/ASM-PINNs>